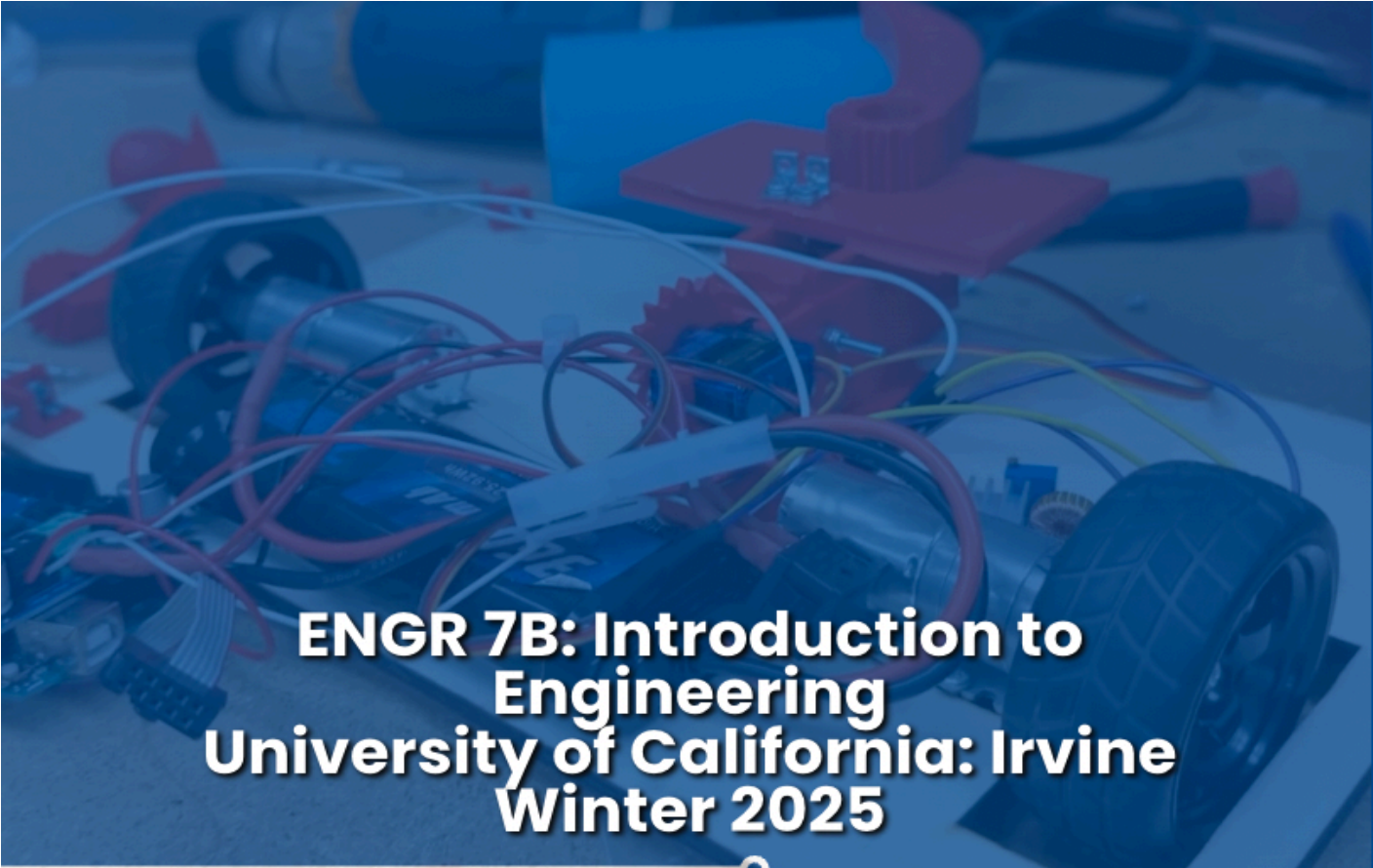




**ENGR 7B: Introduction to
Engineering
University of California: Irvine
Winter 2025**

CRUSTY CRAB FINAL DESIGN REPORT



Ashley Roselynn Vincent
Jennifer Martinez
Sheily Serrano
Danny Bao
Joshua J. Lam
Richard Quoc Ngyuen
Jordan Rui Chan

Table of Contents

Executive Summary	2
Problem Definition	3
Introduction	3
Technical Review / Background	3
Design Requirements	3
Design Description	4
Summary of Design	4
Design Details	5
Wiring Diagram	6
Algorithm Design	7
Action Item Report	10
Task Assignment	10
Gantt Chart	10
Evaluation	11
Calculations	11
Test Plan	12
Results & Discussion	12
Appendix A: SOLIDWORKS Drawings	13
Appendix B: Bill of Materials	31
Appendix C: Arduino Autonomous Mapping Code	32
Appendix D: References	37

Executive Summary

The Engineering 7B course involves working in groups of 6-7 people to design, build, and test an automated rover that can traverse a course, follow a black line to a destination, and then locate a red object and pick it up. Our group aimed to complete the course among the top ten rovers while learning new technical skills such as CAD, coding, manufacturing, wiring, and soldering throughout the quarter.

At the start, we decided that each of us would pick a role where we could gain technical knowledge. We did not pick roles that we had previously in Engineering 7A. Instead, we stepped outside of our comfort zone. One of the first topics that we discussed was designing our rover and the dimensions of it. We wanted to be really creative, but also wanted our rover to work. Our rover would be based on the Krusty Krab, a burger restaurant from Spongebob. We decided on an 12" x 7". These dimensions provided us with a good amount of space for electronics, batteries, and claw. We placed our battery towards the back and the claw towards the front in order to distribute its weight evenly. Next, we thought about the placement of our wheels. Initially, we wanted to put the wheels on the sides of the chassis, but decided it would be better to make 2 holes on our chassis and put the wheels inside of them. We decided to put our IR sensors on the back since it didn't matter if it was placed in the front or back. We wanted to put our pixy cam under or on top of our claw, but it was troublesome since it blocked the claw. Instead, we put it slightly off to the side. This wouldn't really impact our claw since we can program our pixycam to change where the red object should be when our claw opens. We decided the shell and chassis of our rover would be made of laser-cut wood and the claw would be made of 3D print. After we decided on our placements, we started to CAD it.

While using Solidworks, we realized that we had not come up with a design for the claw or the dimensions for it, so we got to work. We chose to build a claw that was similar to one of the examples since it looked really stable and easy to 3D model. One micro servo would manage the lifting mechanism and the other was for the opening mechanism on the claw. The claw was the most challenging since our CAD members did not have any experience. When the claw was done, we began to laser cut and 3d print our parts. We had to reprint some parts due to misprinting and dimension differences while fabricating.

During our fabrication process, since we had trouble with our claw, we had to reCAD and reprint some parts because they were not working as intended. This alone added 4 hours into our budget for 3d printing. Since our screw holes in the CAD were exactly 3mm, we also had to screw them into 3d parts since they were snugly fit. This was good when it came to screwing two 3d printed parts together. Also, we decided on 3 IR sensors for line tracking. Our pixycam was by the claw to help with color tracking.

We began our programming process in week 3, our programming members split up the code with one of them working on the line tracking code and the other working on color tracking. Their initial plan was to do the line tracking through IR sensors and color tracking through the PixyCam, but to experiment line tracking with PixyCam if they had time.

However, testing took longer than anticipated due to faulty IR sensors, arduino board, motor shield driver and motors. Having to replace all these components on the rover multiple times set us back on testing by multiple weeks.

By the end of week 10, our rover successfully completed the preliminary track's line tracking portion and color tracking portion individually. However, when our code was integrated, neither function would work. Unfortunately, since we had run out of time by the end, we could not get a time on the preliminary track for our rover because we were not able to get the code for line tracking and color tracking together to work.

Problem Definition

Introduction

This project aims to design and build an autonomous rover that could follow a line and then locate a blue object and pick it up. The course has sharp turns and curves. The rover has to follow all project requirements as specified by the course instructor while creating a business-type presentation to "promote" our rover.

Our team goal was to learn new technical skills while building our rover, from CAD to coding. Additionally, we set a goal for our rover to place within the top ten teams by completing the course within a short time.

Technical Review / Background.

Rovers were first developed for space exploration, with Lunokhod 1 in 1970 being the first successful robotic rover on the Moon. Over time, rovers became more advanced, with NASA's Curiosity and Perseverance exploring Mars using artificial intelligence, strong wheels, and advanced scientific tools. These space rovers inspired new uses on Earth, including robots that help in the food industry. Today, companies are using rovers to deliver food, manage inventory in warehouses, and even assist in farming. Food delivery rovers, such as those from Starship Technologies and Nuro, use AI and cameras to move safely through streets, delivering meals and groceries. Some food robots are used in restaurants or warehouses to transport ingredients, while others help with farming by planting and harvesting crops. Many food rovers include temperature control to keep food fresh, long-lasting batteries for efficiency, and sanitization features to ensure hygiene.

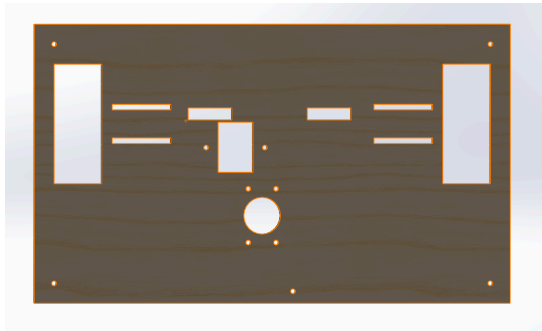
By studying different food delivery and warehouse robots, we learned important lessons that helped guide our design. For example, we saw that navigation and safety are key, so we focused on using AI and sensors to help our rover move safely. Looking at food delivery robots, we realized the need for temperature control, leading us to explore insulation and cooling systems. Studying warehouse and farm robots showed us the importance of long battery life, so we focused on energy-efficient power options. We also learned that customers prefer easy-to-use robots, so we considered adding touch screens or app controls for a smooth experience. Finally, hygiene is very important in the food industry, so we explored ways to make our rover self-cleaning or easy to sanitize.

Design Requirements

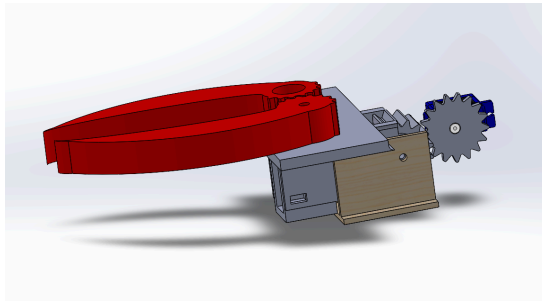
The design requirements for our rover were smaller than 12x16 inches which include the wheels and claw mechanism. We can only use materials in the student parts list and 3D printing and laser cutting were allowed. Our battery had to be easily removable as well. The cost requirement was less than \$300 and must be broken down in the Bill of Materials. It should be able to follow the black line on a course and be able to lift an object.

Design Description

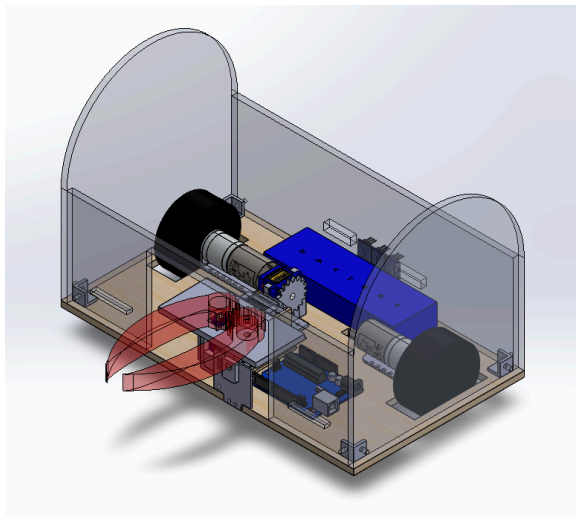
Summary of Design



This is our chassis and we have various cuts in it for the wheels, motors, castor ball, and micro servo.

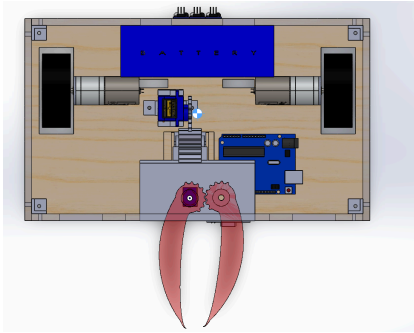


This is our claw mechanism and it will be connected to our chassis. It will be used to pick up an object.

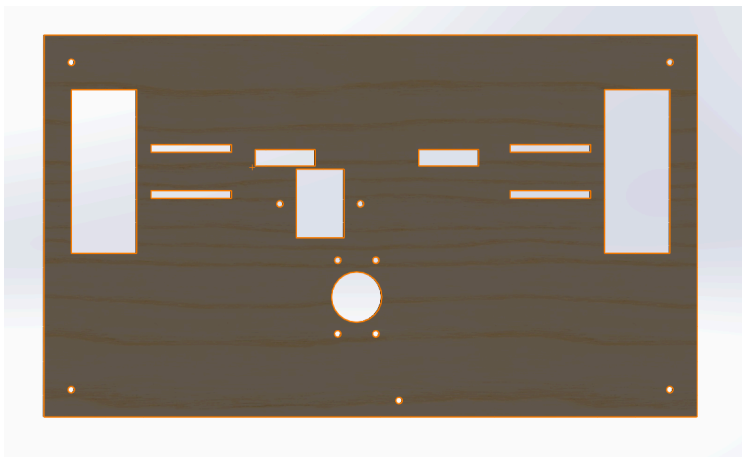


This is the entire CAD design. The shell makes it look like the Crusty Crab. We put the battery in the back and the Arduino towards the front to spread out the weight of our rover. The infrared sensors will be in the back.

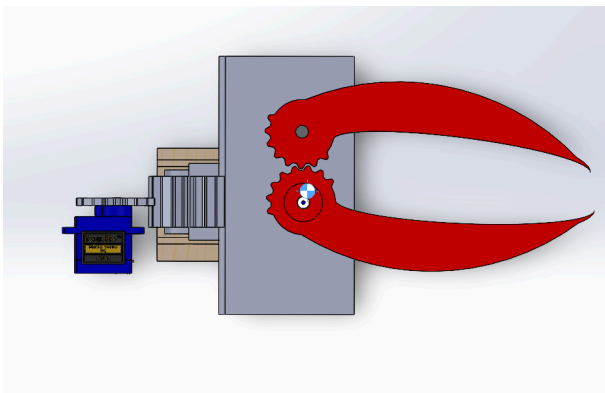
Design Details



Our center of mass will be in the center which will make sure that the rover will not tip over.

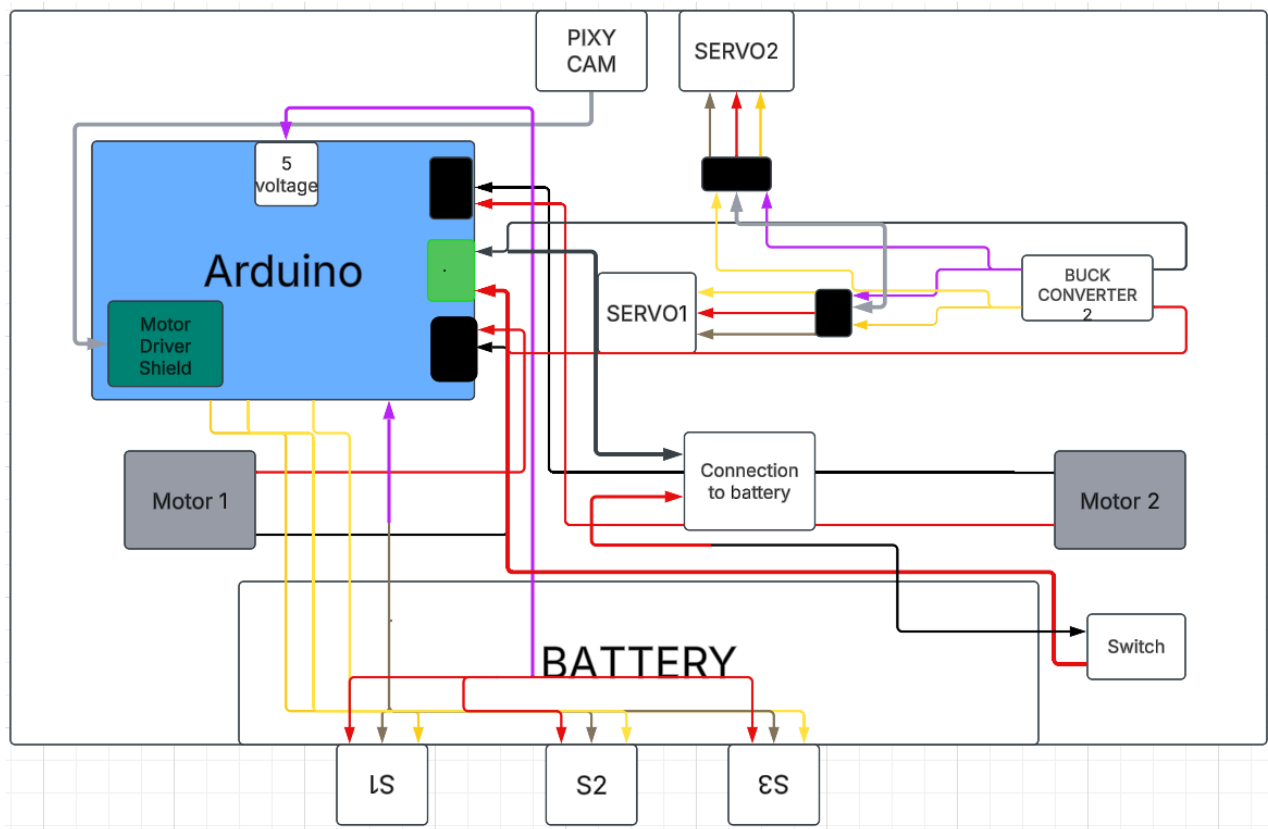


The two big rectangle cuts are for the wheels. The 2 rectangles by each big rectangle are for the motor bracket. The circular cut is for the castor ball. The rectangle in the middle is for our micro servo. The cut above is for the velcro to hold the battery. It is a 7"x12" chassis.



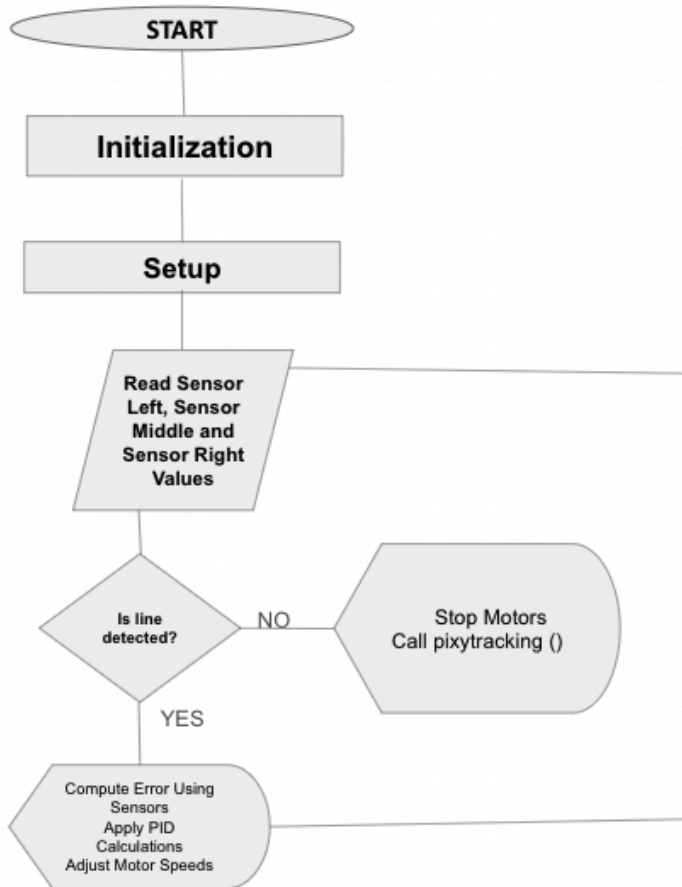
This is the center of mass for our claw. Our claw was supposed to even out the weight between the battery so that our center of mass will be in the center of our chassis.

Wiring Diagram

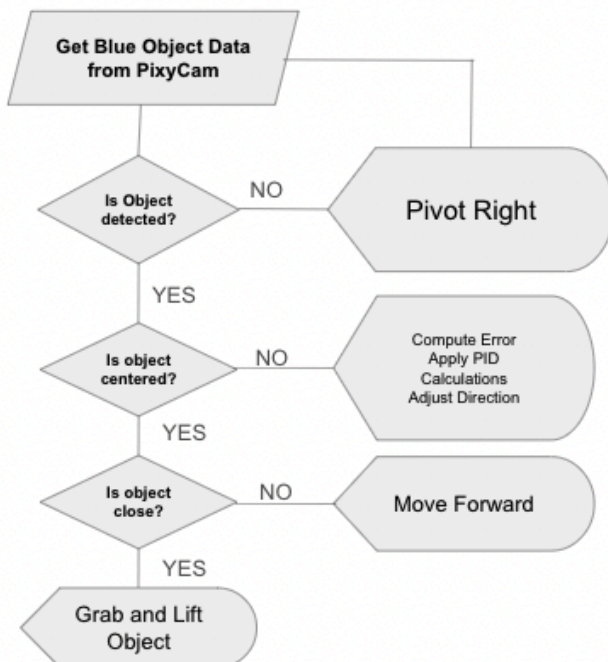


Algorithm Design

Flow Chart:



pixytracking() :



Pseudocode:

INITIALIZATION PHASE

Include External Libraries
(Motor, PixyCam, and Servo)

Initialize constant variables with appropriate data type and designate pin #s.
(PixyCam coordinates grid, PID constants, speed, and turn thresholds)
//Set line tracking constants with lowercase and color tracking constants with uppercase

Create objects for PixyCam and Color Tracking

SETUP PHASE

Define sensors as input values
Initialize base claw level, start Pixycam, and designate servo pins

LOOP PHASE

Boolean statement: (True) -> If all sensors detect white continue
if (False)
Stop all motor movement
Begin PixyCam Tracking -> Call PixyCam function
return value //repeatedly awaiting final line detect

initialize error variable for line tracking //line tracking error

if (Middle sensor detects line or sees black): Error = 0
if (Left sensor detects line or sees black) Error = 1
if (Right sensor detects line or sees black) Error = -1

Add error values to itself = integral gain
Subtract total error from most recent error = derivative gain
(Proportional gain constant x error) + (Integral constant x integral gain) + (derivative constant x derivative gain) = correction value //based on PID theory and formula

Left motor speed = Base speed - correction value
Right motor speed = Base speed + correction value

Set motor speed with (Left or Right motor speed, respectively)

delay

Loop end

PixyCam function
if error > turn threshold constant
call left turn function (speed scope)

```

    else if error < -turn threshold constant
    call right turn function (speed scope)

    else
    if (object width constant is >= target width constant)
        call grab function
    else
        call move forward function

else
call right turn function

Tracking Loop
call track function

Motorstop function
    set motor speeds = 0

grab function
    call motorstop function
    delay
    servo command
    delay
    servo command

move forward function
    set equivalent motor speeds

right turn function
    right motor = speed
    left motor = -speed

left turn function
    right motor = -speed
    left motor = speed

track function
    call library pixy cam function "get blocks"
    if (pixy cam function >= 1)
    create variables
    object x value
    object width value
    error = object x value - center x value constant
    output = proportional gain constant x absolute value of error
    speed scope = constrain upper and lower bounds

```

Action Item Report

Task Assignment

Our team, Crusty Crab, was tasked with making an autonomous rover that could do line tracking and then pick up an object with a claw through a pixy cam. This project required many stages for creating our ideal rover including making a CAD model, fabricating, assembling, and testing. All of our team members were responsible for specific roles and responsibilities. Each member worked outside of the lab to speed up the process.

Our team captain, Ashley, was in charge of our Gantt chart and PO forms to ensure that our team could complete tasks in a timely manner.

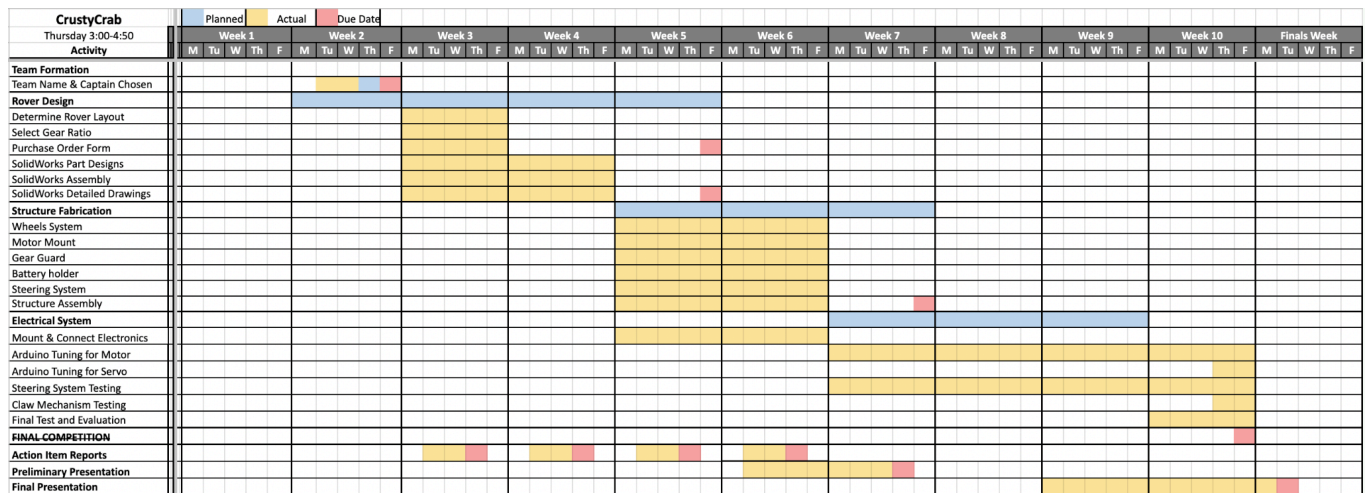
Joshua and Danny were in charge of our CAD model and created our claw and chassis while updating it if the electronics were moved around.

Sheily, Jordan, and Jennifer were in charge of the electronics and they soldered our electronics.

Ashley and Richard were in charge of the code and they worked to complete the line tracking and color tracking code.

We completed many tasks during the quarter from selecting our wheel size to deciding where our infrared sensors would be. We completed the Solidworks design by week 4, assembled it by week 7, and fixed it by week 10.

Gantt Chart



Evaluation

Calculations

Weight of Vehicle:

$$\text{Weight} = \text{mass} \times \text{gravity} = 1.3857 \text{ kg} \times 9.8 \text{ m/s}^2 = 13.580 \text{ N} \approx \mathbf{3.05 \text{ lbs.}}$$

Predicted Drive Time:

$$\text{Motor Current} = \frac{7.2\text{V} \times 5\text{A}}{12\text{V}} = 3\text{A}$$

$$\text{Battery Capacity} / \text{Total Current} = \frac{3.6\text{Ah}}{\text{Motor Current} + \text{Servo Current}} = \frac{3.6\text{Ah}}{3\text{A} + 3\text{A}} = 0.6 \text{ h}$$

$$0.6 \text{ h} / 60 \text{ min.} = \mathbf{36 \text{ min.}}$$

Mechanical Advantage:

$$\mathbf{34:1 \text{ Gearmotor} = 34}$$

Stall Torque of Electric Motor:

$$\text{Torque}_{\text{output}} = \frac{7.2\text{V} \times 11\text{kg}\cdot\text{cm}}{12\text{V}} = 6.6 \text{ kg}\cdot\text{cm}$$

$$\text{Torque}_{\text{output}} = 6.6 \text{ kg}\cdot\text{cm} \times 9.81 \text{ m/s}^2 \times \frac{1 \text{ m}}{100 \text{ cm}} = 0.647 \text{ N}\cdot\text{m}$$

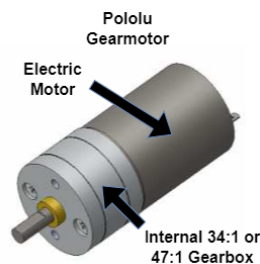
$$\text{Stall Torque}(\text{Torque}_{\text{input}}) = \text{Torque}_{\text{output}} / \text{Mechanical Advantage} = 0.647 \text{ N}\cdot\text{m} / 34 = \mathbf{0.019 \text{ N}\cdot\text{m}}$$

Stall Weight

$$\text{Stall Weight} = \frac{\text{Stall Torque}}{\text{Radius}_{\text{wheel}}} = \frac{0.019 \text{ N}\cdot\text{m}}{0.0325 \text{ m}} = 0.585 \text{ N} \approx \mathbf{0.132 \text{ lbs.}}$$

Motor Choice:

Each team had an option to choose either the 47:1 gearmotor or the 34:1 gearmotor. We decided to choose the 34:1 gearmotor to optimize speed over power for our rover. If we chose the 47:1 gearmotor, our rover would have leaned towards more torque than speed because of its mechanical advantage being higher. This would also mean that this would add stall weight to our rover.



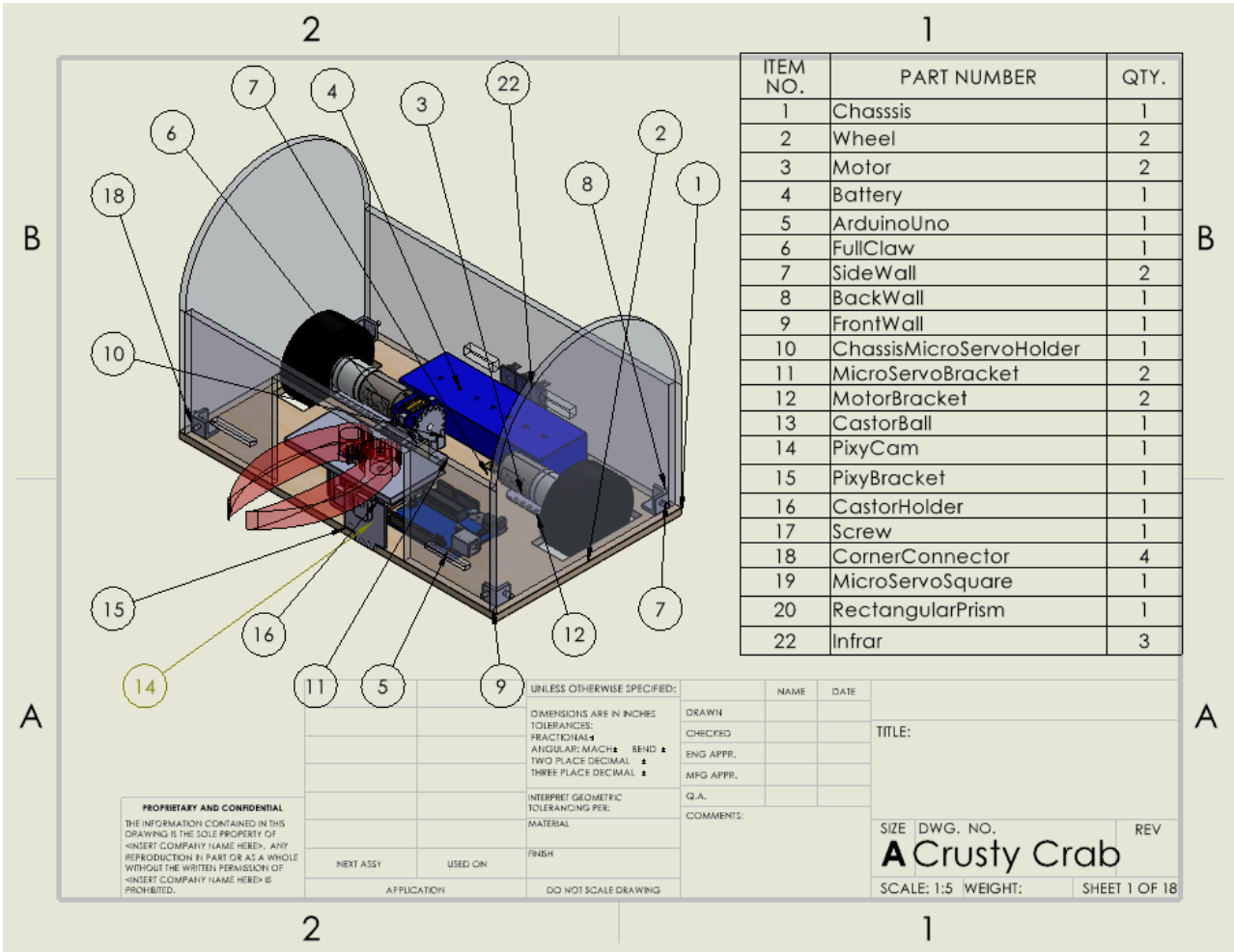
Test Plan

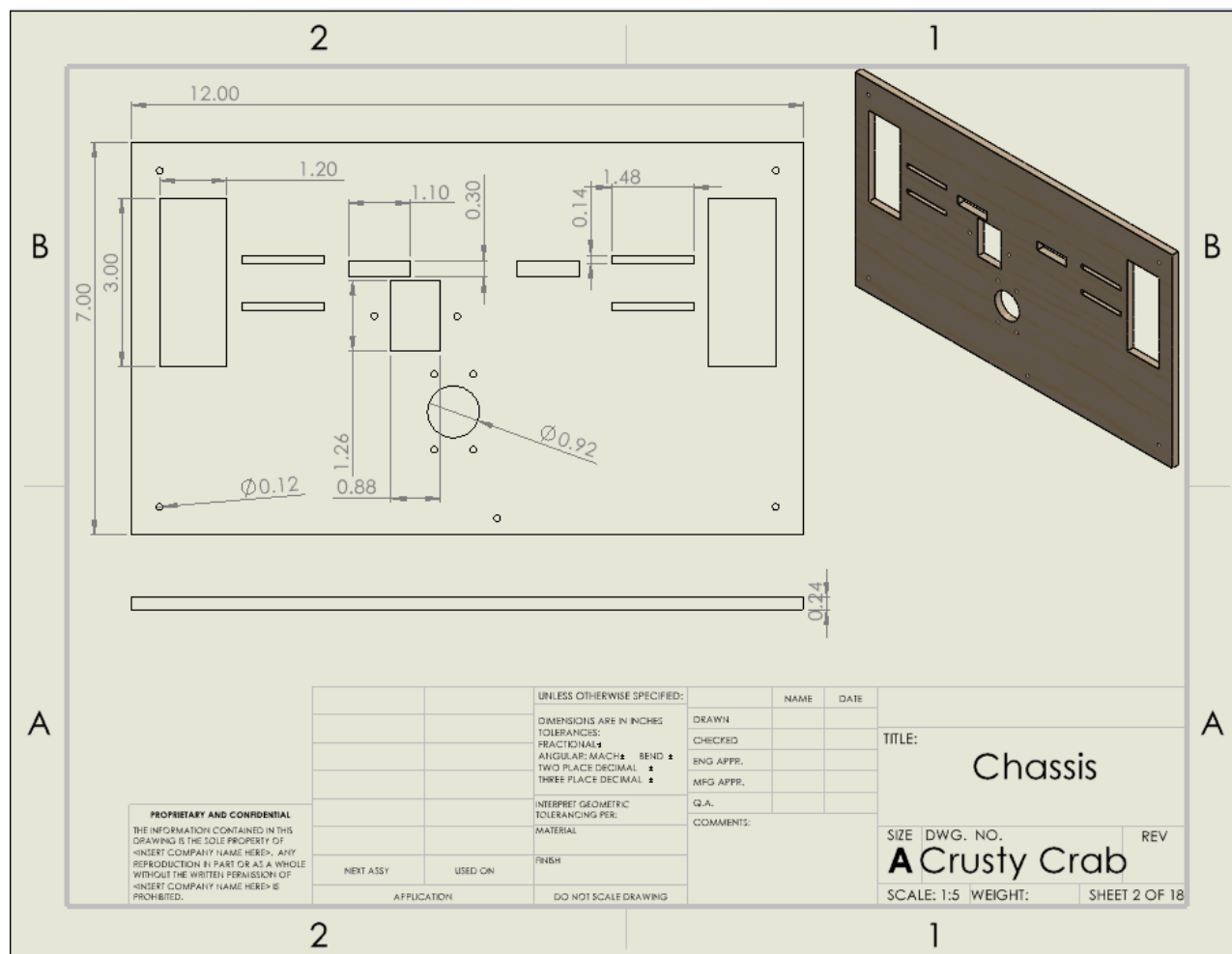
Our first step in assembling the rover was mounting the motors. To ensure they functioned properly, we attached them to the wheels and tested their connection using an Arduino. Once the wheels began moving as expected, we confirmed that the motors were working correctly. After verifying the motors, we proceeded with developing and testing the code for the infrared (IR) sensors. The IR sensors needed to detect black surfaces accurately in order for it to properly function. Initially, the sensors did not respond as intended, so we modified the code to improve their sensitivity and ensure they could correctly differentiate between surfaces. While the IR code was being refined, we simultaneously worked on the Pixy Cam to enhance its object and color detection capabilities. Our team encountered issues with color recognition, requiring adjustments to the code to improve accuracy. After several modifications and tests, we were able to resolve these problems and achieve a good performance. Once both the IR sensor and Pixy Cam codes were successfully developed and optimized, we combined them to test their compatibility. This final step ensured that both systems worked together seamlessly, allowing the rover to navigate effectively while detecting and responding as intended.

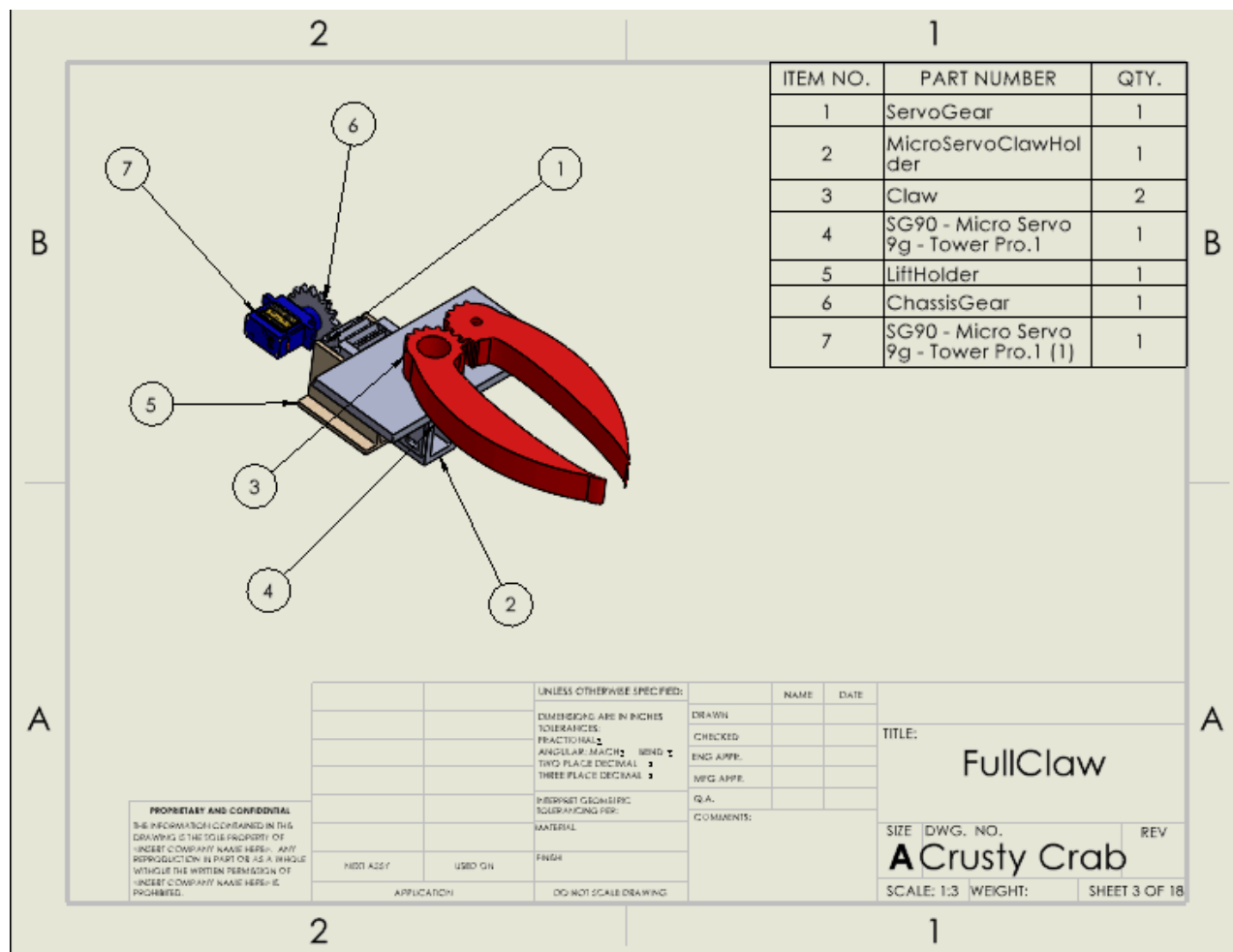
Results & Discussion

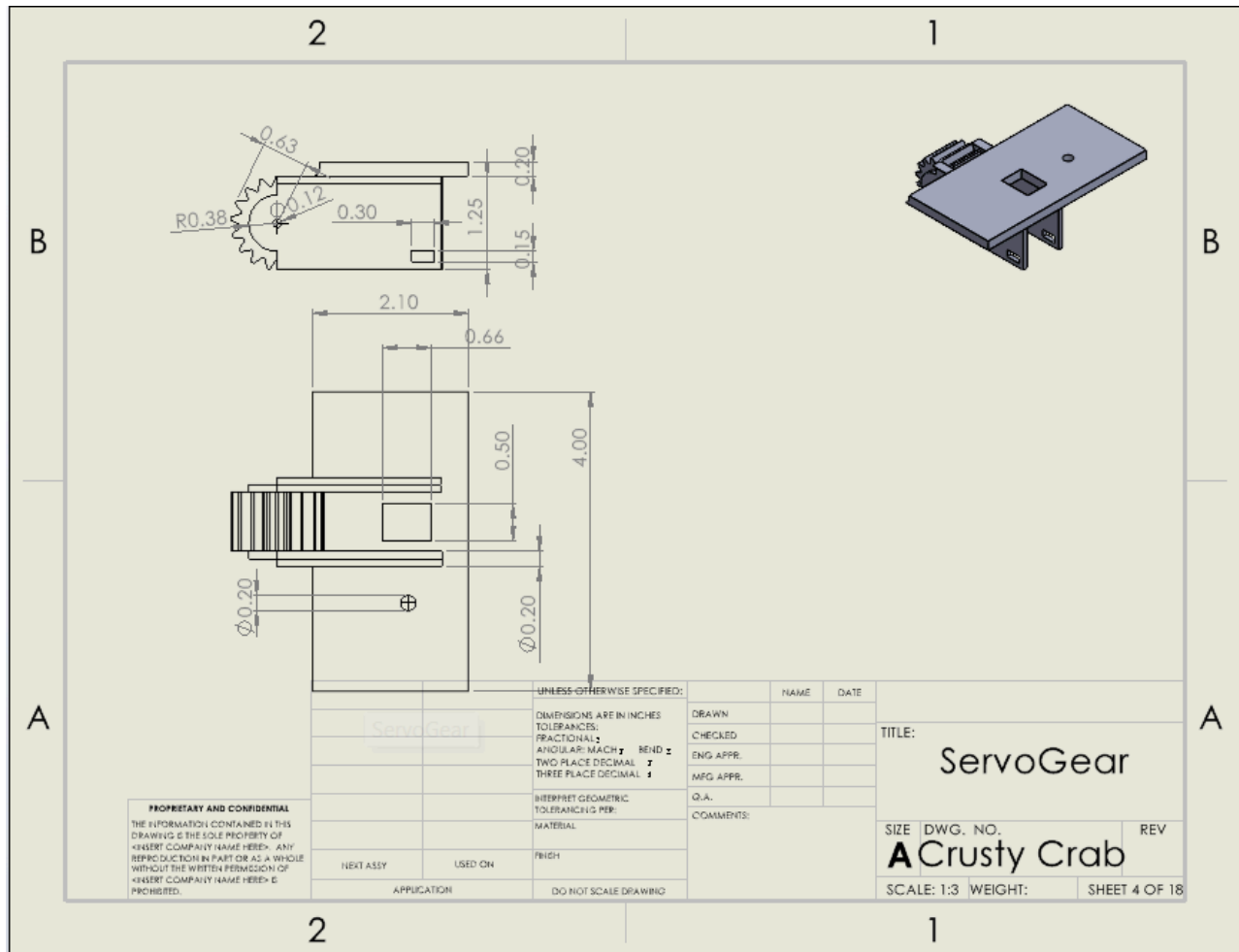
The rover measures 12 by 7 inches and was designed to accommodate all electronic components in an organized manner. One of the challenges we encountered was its low ground clearance, which initially caused movement issues. To fix this, we replaced the screws at the bottom with smaller screws, allowing the rover to move smoothly without problem. To ensure balance and stability, we carefully distributed the electronics evenly across the rover's frame. This helped maintain a well-balanced weight distribution, preventing tilting or uneven movement. During testing, the rover showed smooth movement and responsiveness, but we noticed occasional delays in sensor detection, which affected the rovers navigation. We determined that adjusting the sensor code and improving motor control could significantly enhance the rover's movement and responsiveness. One of the biggest lessons our team learned was the importance of better communication and coordination during team meetings. At times, miscommunication led to delays and confusion, which could have been avoided if we just communicated with one another. Ensuring that every team member was on the same page would have improved efficiency and teamwork. Another key takeaway was the need to start testing the code much earlier. Initially, we believed we had plenty of time, but this assumption ended up hurting us in the long run. As deadlines approached, we had to rush our testing and adjustments. If we had started earlier, we could have refined the code gradually and addressed issues more effectively. If we took this class again we would prioritize testing the code earlier to avoid last-minute adjustments.

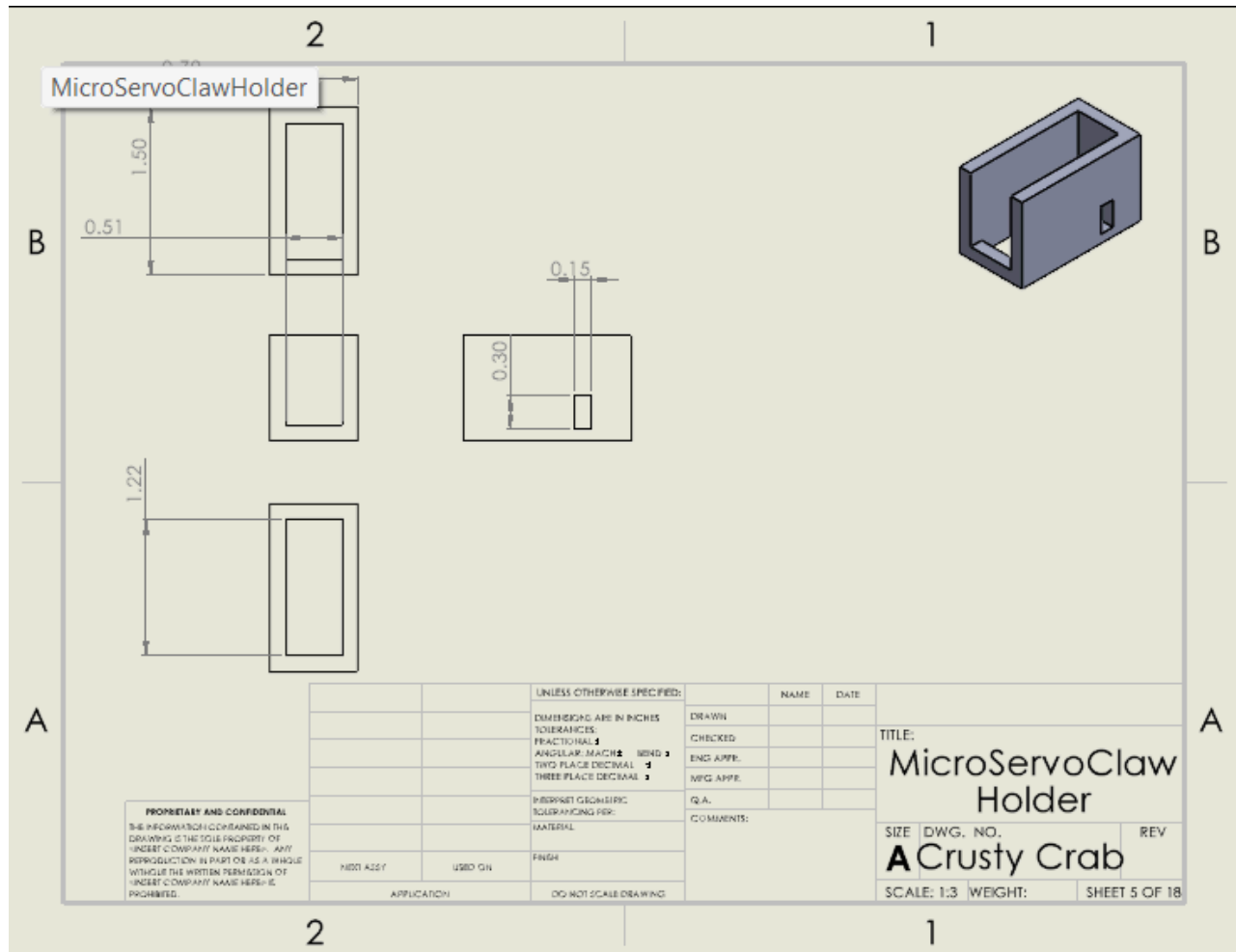
Appendix A: SOLIDWORKS Drawings (Includes BOM)

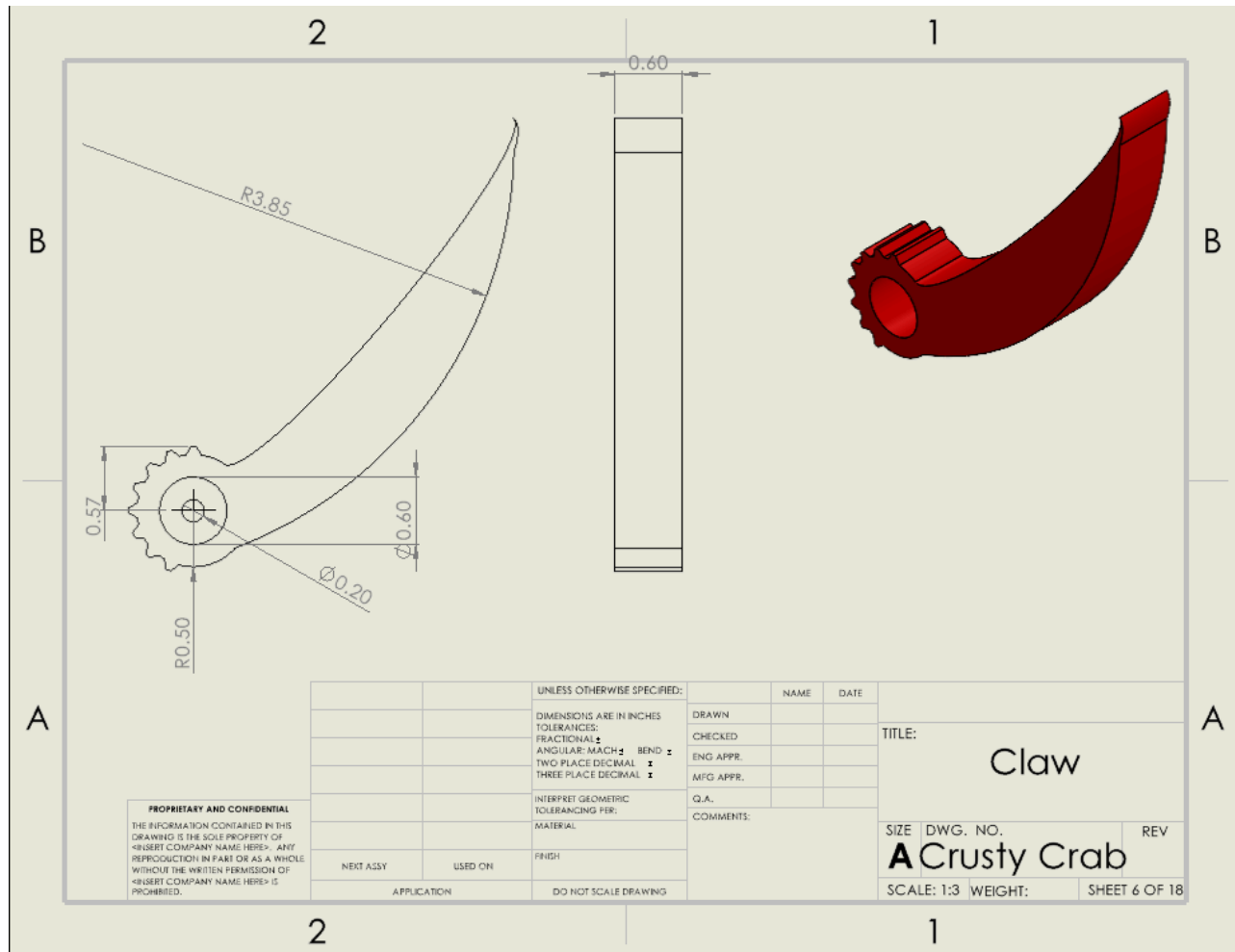


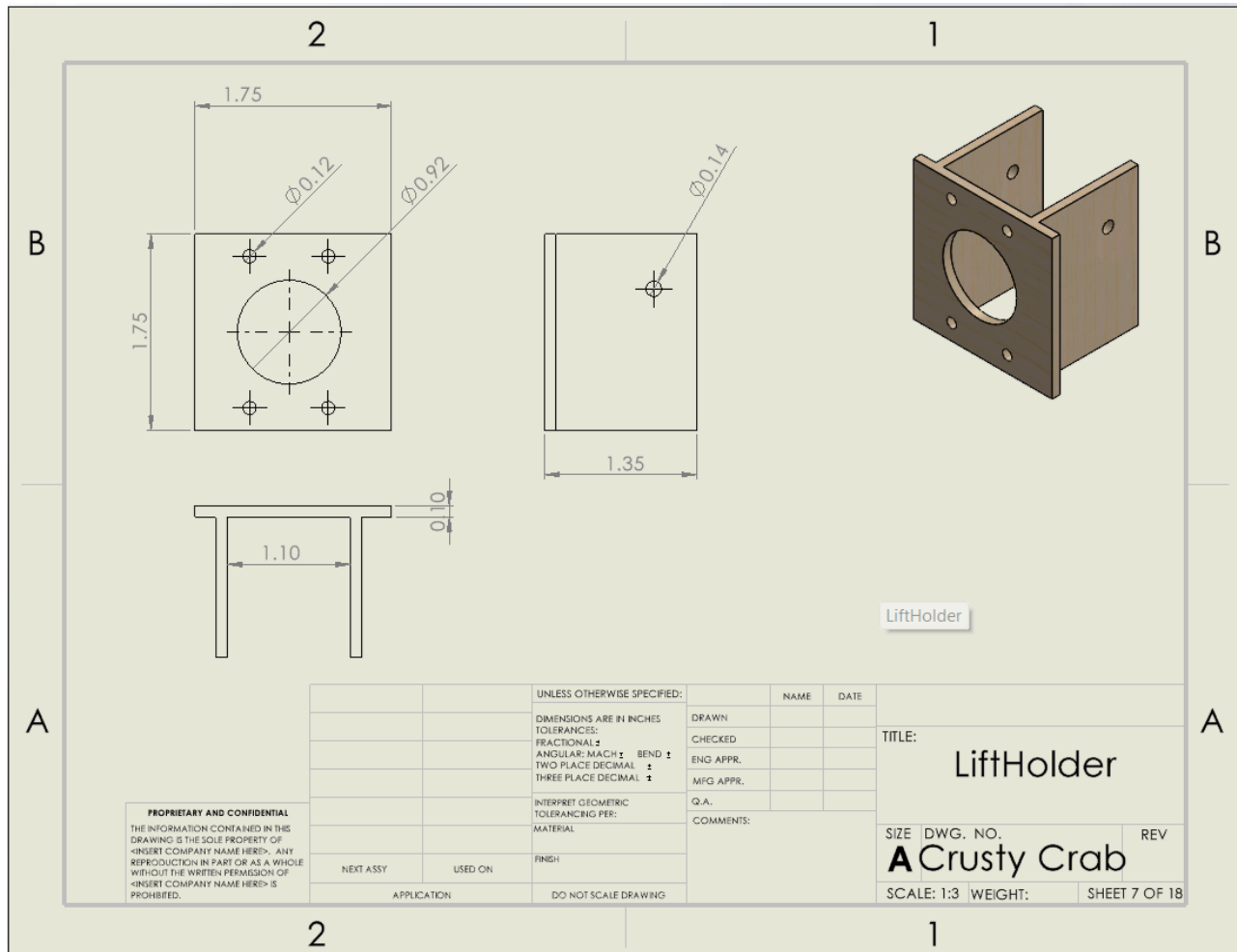


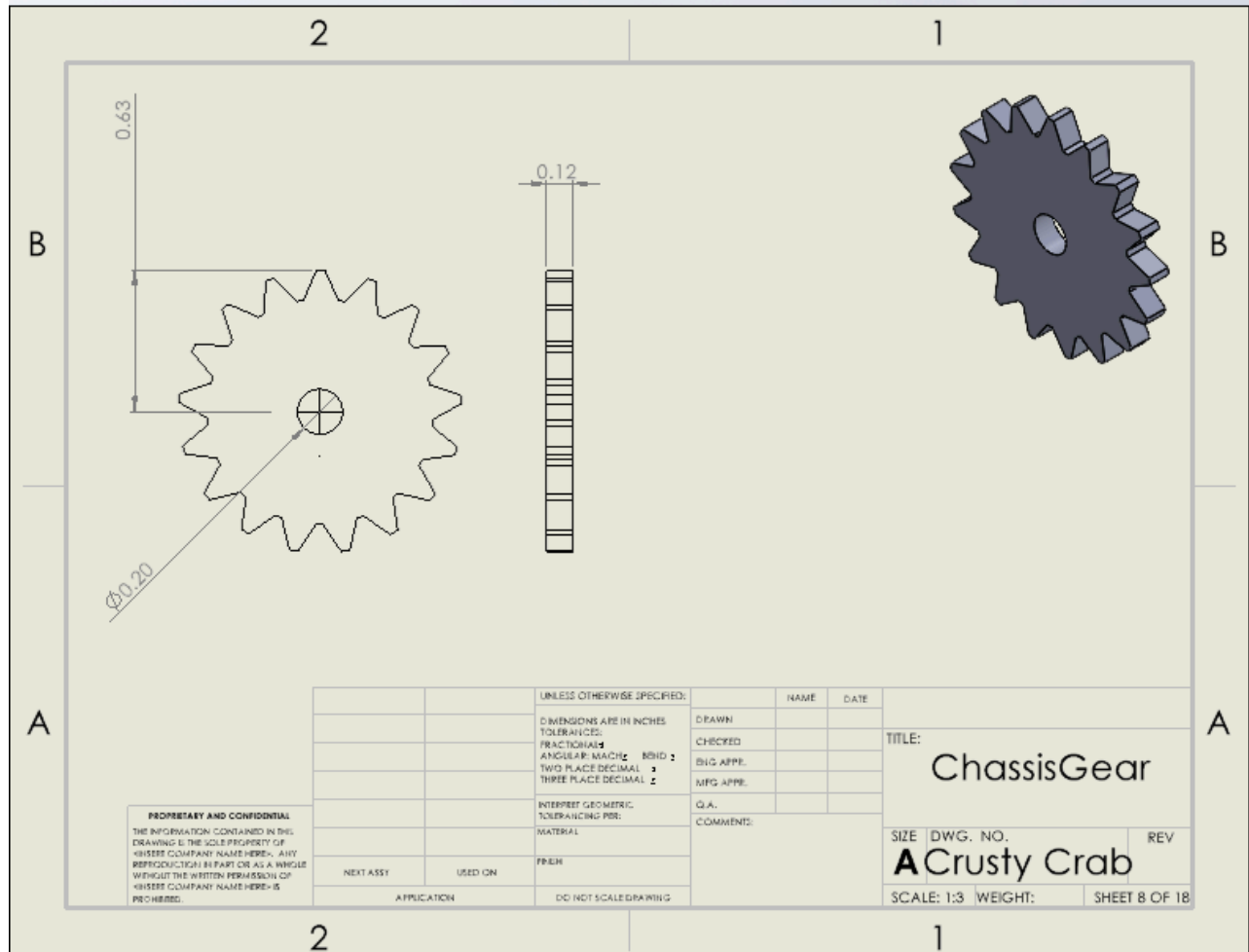


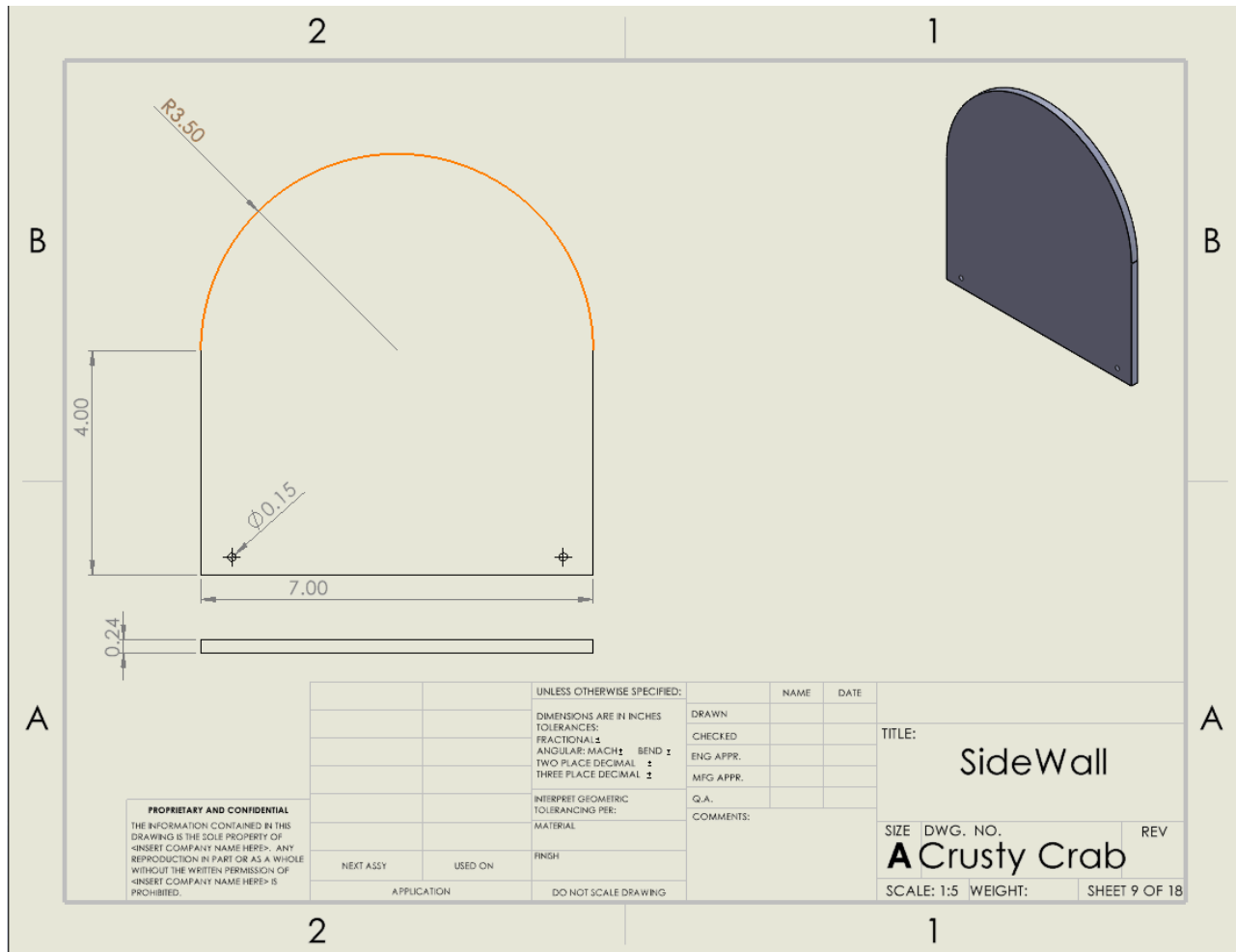


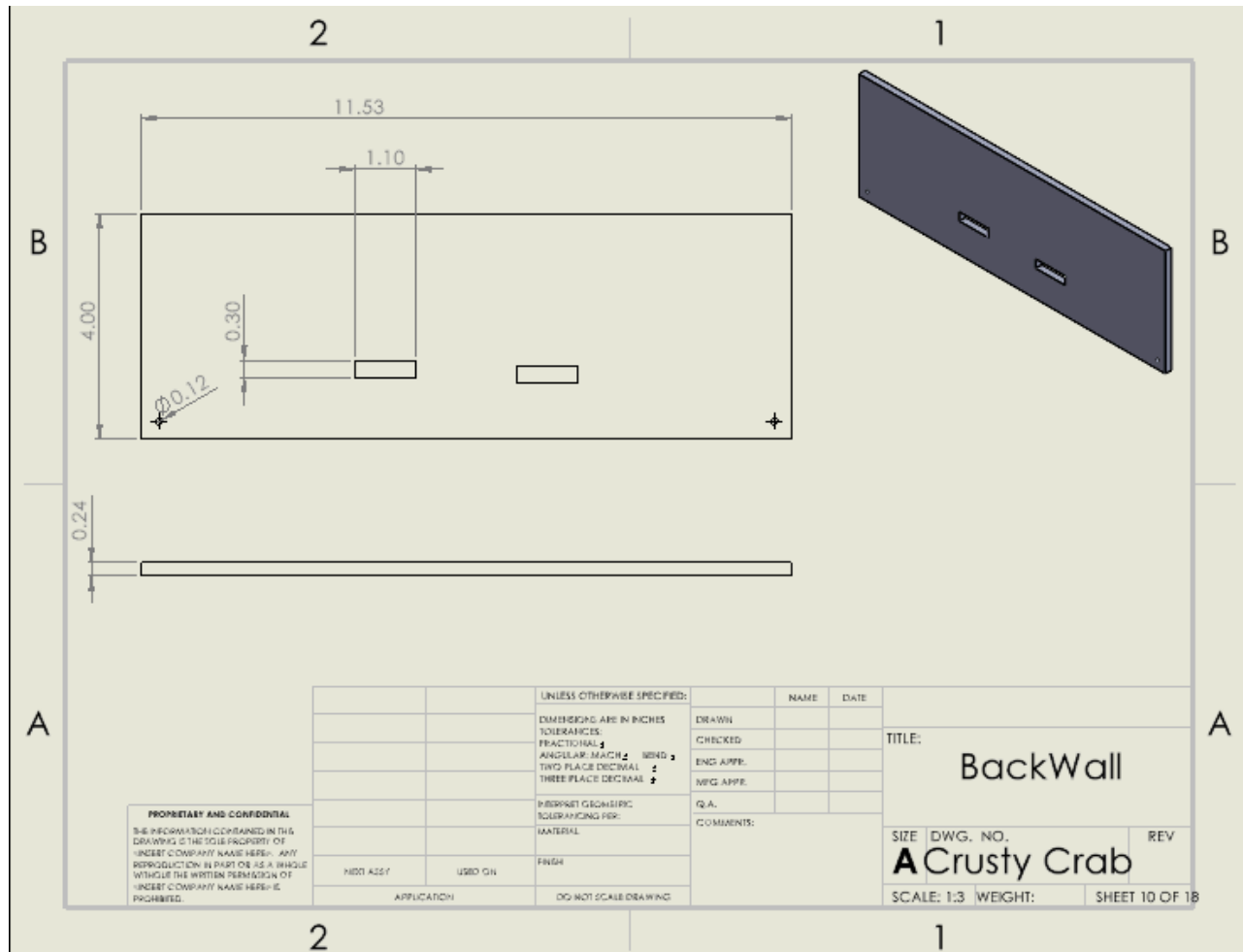


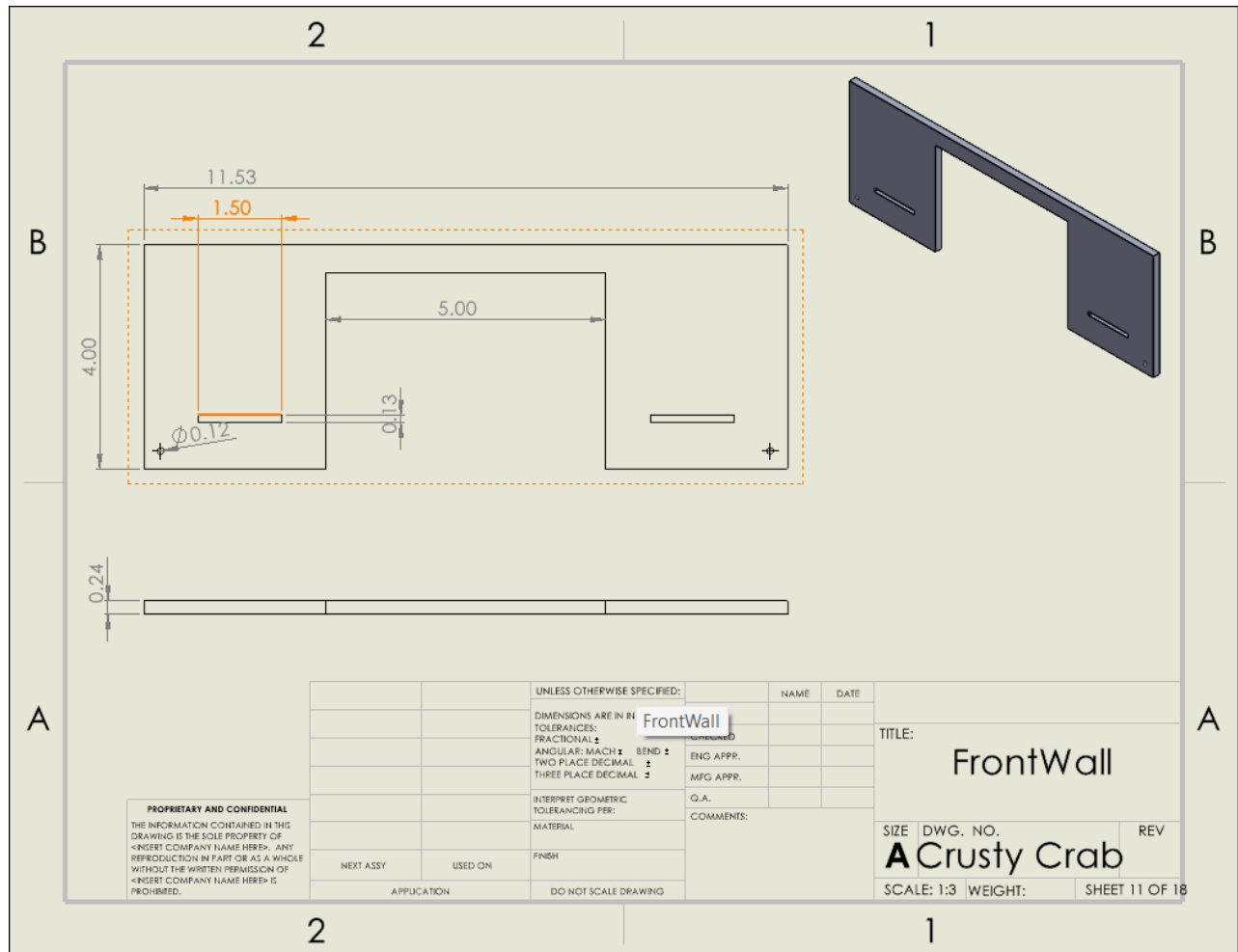


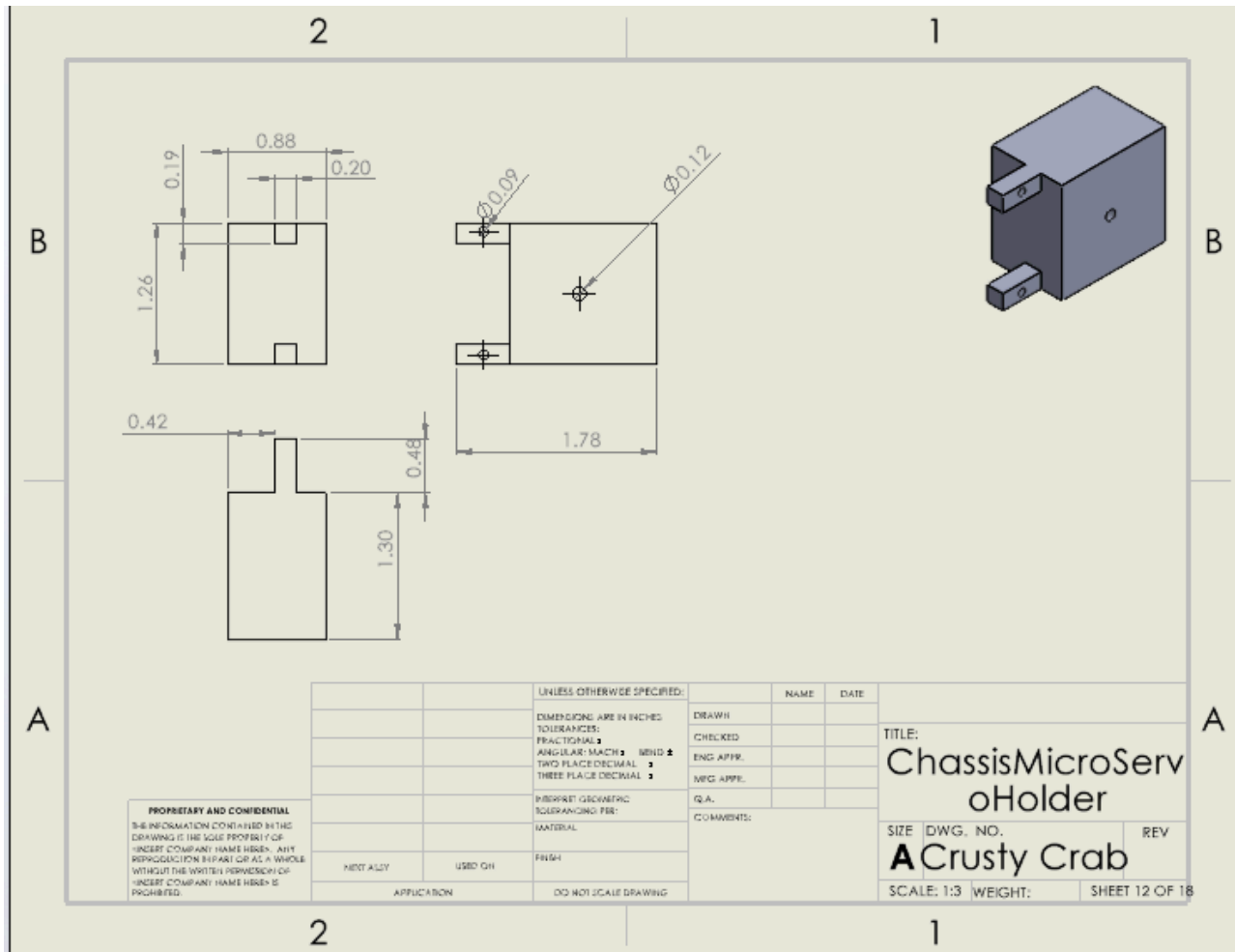


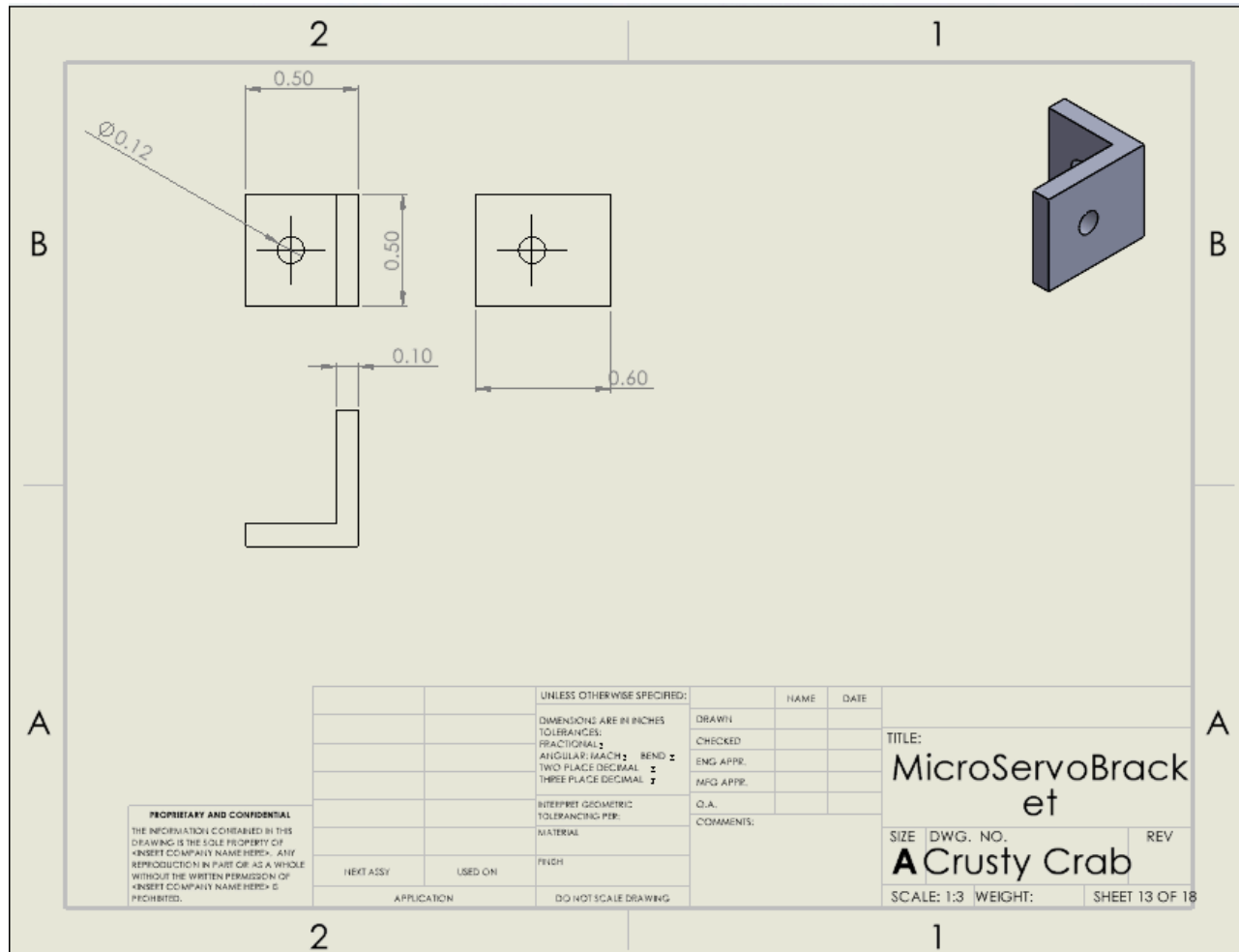


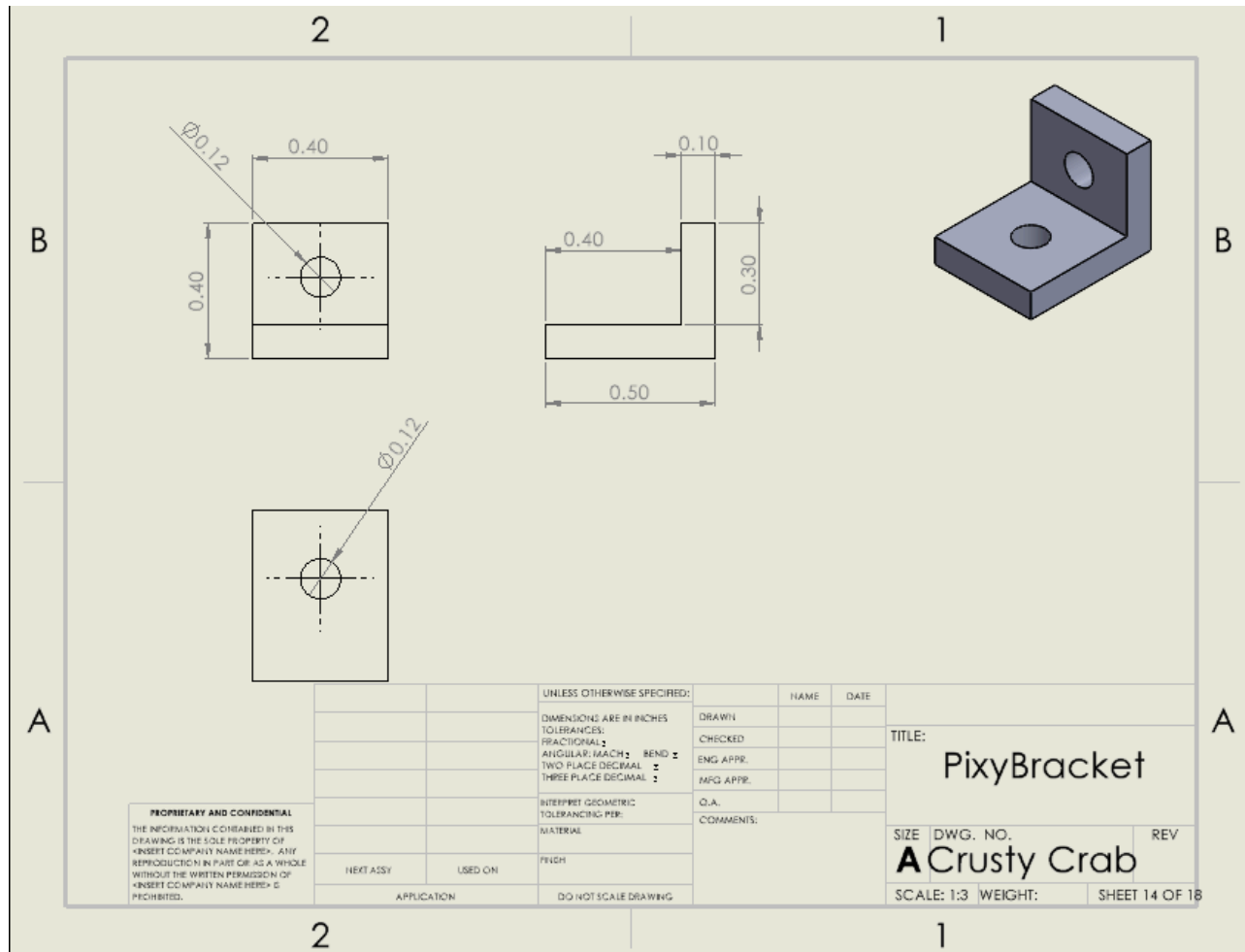


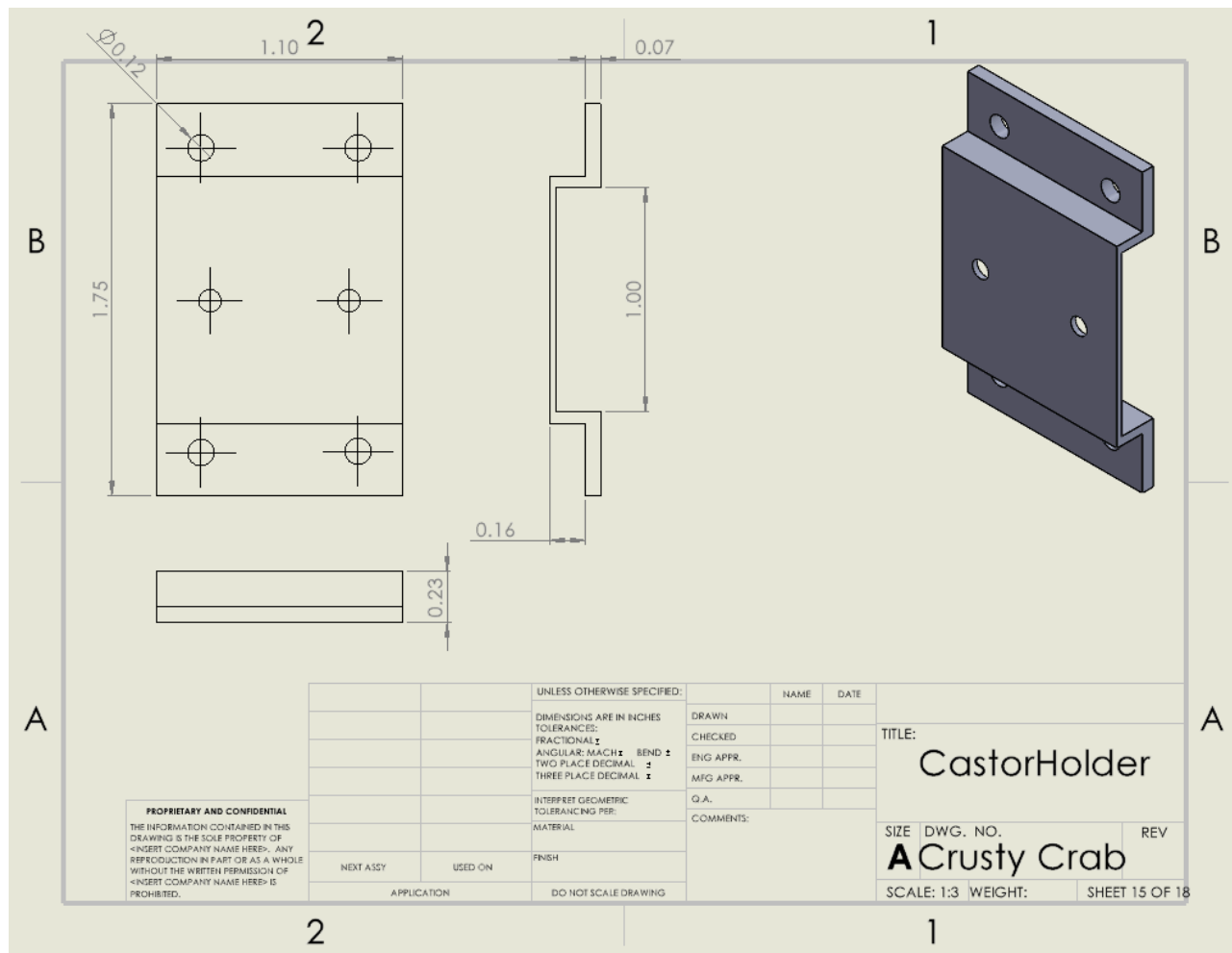


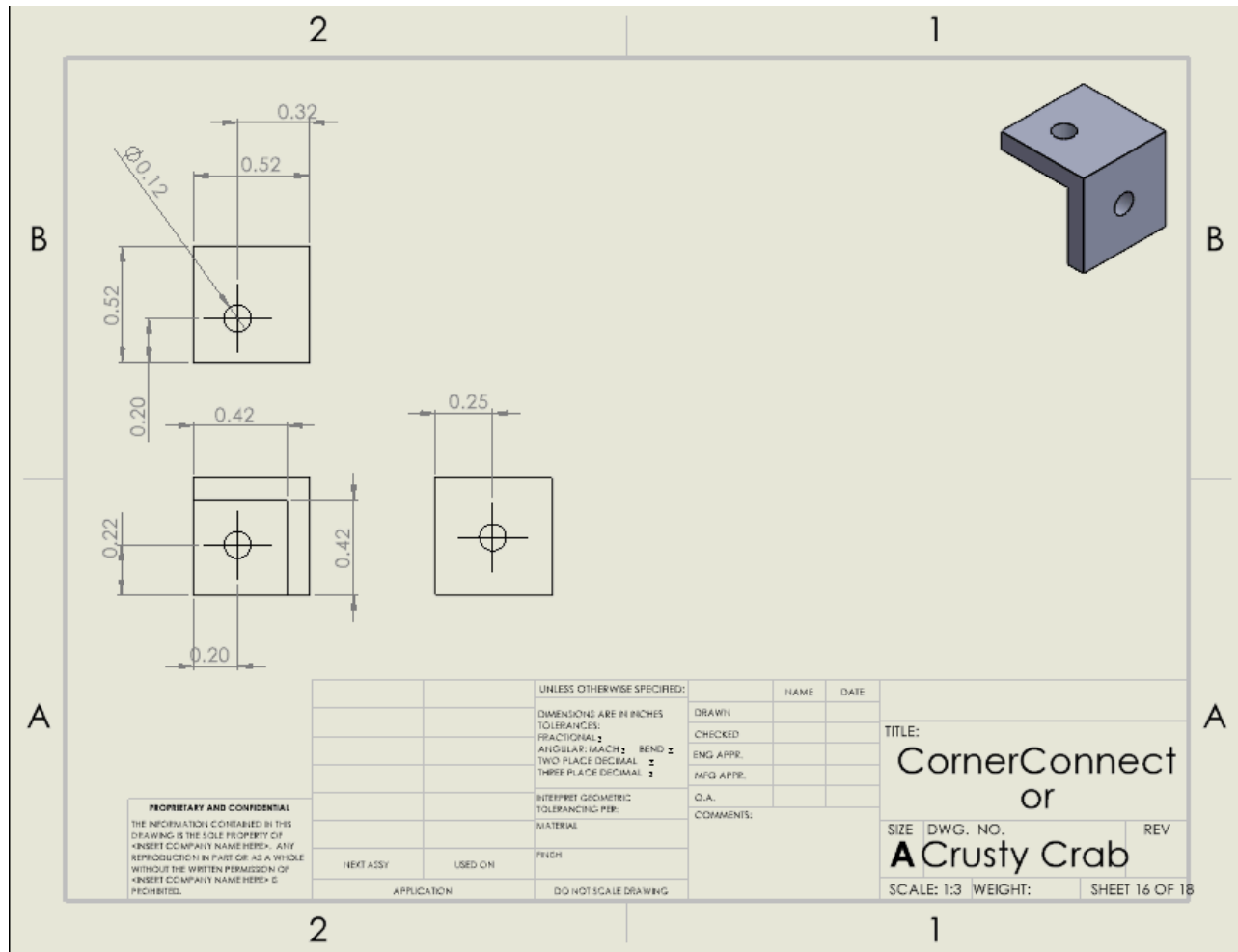


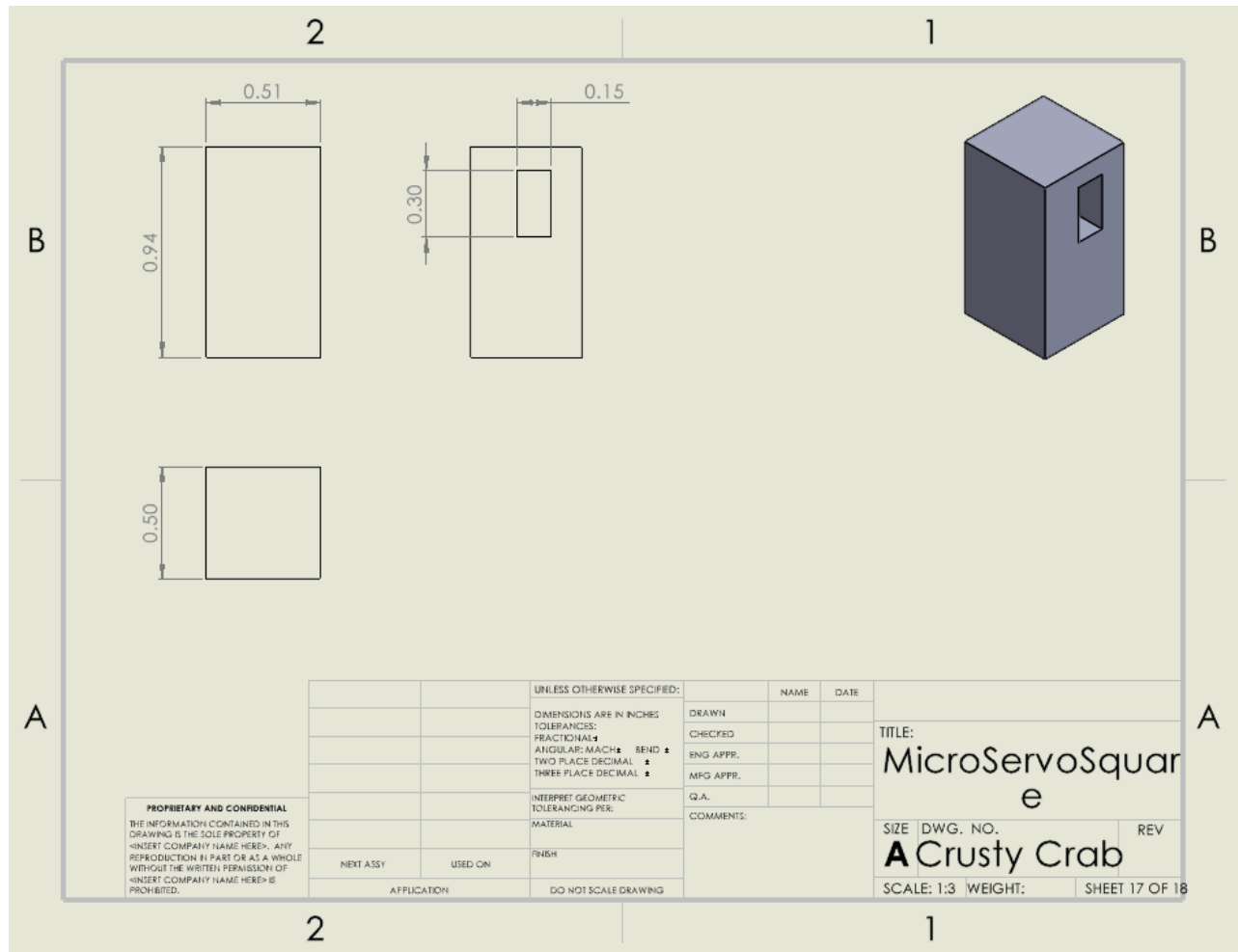


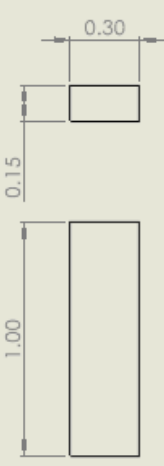
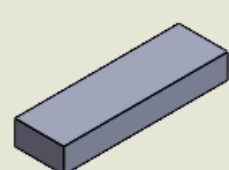










2		1																																															
																																																	
PROPRIETARY AND CONFIDENTIAL THE INFORMATION CONTAINED IN THIS DRAWING IS THE SOLE PROPERTY OF <SHEET COMPANY NAME HERE>. ANY REPRODUCTION IN PART OR AS A WHOLE WITHOUT THE WRITTEN PERMISSION OF <SHEET COMPANY NAME HERE> IS PROHIBITED.	<table border="1" style="width: 100%; border-collapse: collapse;"> <tr> <td colspan="2" style="text-align: center;">UNLESS OTHERWISE SPECIFIED:</td> <td style="width: 10%;">NAME</td> <td style="width: 10%;">DATE</td> </tr> <tr> <td style="width: 30%;">DIMENSIONS ARE IN INCHES</td> <td style="width: 30%;"></td> <td style="width: 10%;">DRAWN</td> <td style="width: 10%;"></td> </tr> <tr> <td>TOLERANCES:</td> <td></td> <td>CHECKED</td> <td></td> </tr> <tr> <td>FRACTIONAL: $\frac{1}{16}$</td> <td></td> <td>ENG APPR.</td> <td></td> </tr> <tr> <td>ANGULAR: MAXIMUM 30°</td> <td></td> <td>MFG APPR.</td> <td></td> </tr> <tr> <td>TWO PLACE DECIMAL: 0.01</td> <td></td> <td>Q.A.</td> <td></td> </tr> <tr> <td>THREE PLACE DECIMAL: 0.001</td> <td></td> <td>COMMENTS:</td> <td></td> </tr> <tr> <td>INTERPRET GEOMETRIC TOLERANCING PER:</td> <td></td> <td></td> <td></td> </tr> <tr> <td>MATERIAL:</td> <td></td> <td></td> <td></td> </tr> <tr> <td>FINISH:</td> <td></td> <td></td> <td></td> </tr> <tr> <td>NEXT ASSY</td> <td>USED ON</td> <td></td> <td></td> </tr> <tr> <td colspan="2" style="text-align: center;">APPLICATION</td> <td colspan="2" style="text-align: center;">DO NOT SCALE DRAWING</td> </tr> </table>	UNLESS OTHERWISE SPECIFIED:		NAME	DATE	DIMENSIONS ARE IN INCHES		DRAWN		TOLERANCES:		CHECKED		FRACTIONAL: $\frac{1}{16}$		ENG APPR.		ANGULAR: MAXIMUM 30°		MFG APPR.		TWO PLACE DECIMAL: 0.01		Q.A.		THREE PLACE DECIMAL: 0.001		COMMENTS:		INTERPRET GEOMETRIC TOLERANCING PER:				MATERIAL:				FINISH:				NEXT ASSY	USED ON			APPLICATION		DO NOT SCALE DRAWING	
UNLESS OTHERWISE SPECIFIED:		NAME	DATE																																														
DIMENSIONS ARE IN INCHES		DRAWN																																															
TOLERANCES:		CHECKED																																															
FRACTIONAL: $\frac{1}{16}$		ENG APPR.																																															
ANGULAR: MAXIMUM 30°		MFG APPR.																																															
TWO PLACE DECIMAL: 0.01		Q.A.																																															
THREE PLACE DECIMAL: 0.001		COMMENTS:																																															
INTERPRET GEOMETRIC TOLERANCING PER:																																																	
MATERIAL:																																																	
FINISH:																																																	
NEXT ASSY	USED ON																																																
APPLICATION		DO NOT SCALE DRAWING																																															
2		1																																															

B

B

A

A

TITLE:
 Rectangular Prism

SIZE DWG. NO. REV
 A Crusty Crab

SCALE: 1:3 WEIGHT: SHEET 18 OF 18

Appendix B: Bill of Materials

Quantity*	# of Units in package *	Company*	Item Description*	Catalog #*	Price*	Estimated Extended Price*
1	1	Arduino	Arduino Uno	A000066	\$27.60	\$27.60
1	1	Cytron	Motor Driver Shield	SHIELD-MDD10	\$26.00	\$26.00
1	4	Amazon	Buck Converter	B079N9BFZC	\$13.99	\$3.50
1	3	Amazon	Male Tamiya Connector	B07VL2B5C8	\$8.69	\$2.90
1	10	Amazon	Power Switch	B071Y7SMVQ	\$4.99	\$0.50
1	2	Amazon	Battery	B07VLKP6RJ	\$34.99	\$17.50
1	1	Adafruit	PixyCam 2	1906	\$74.95	\$74.95
3	20	Amazon	IR Sensors	B08DR1W3BK	\$9.99	\$1.50
2	12	Amazon	SG90 Microservo	B08FJ27Q1H	\$16.99	\$2.83
2	1	Pololu	34:1 Gearmotor	3204	\$28.95	\$57.90
4	4	Amazon	65mm x 26mm Wheels	10205648	\$10.99	\$10.99
2	2	Pololu	Hex Wheel Adapter	2684	\$4.95	\$4.95
1	1	Pololu	3/4" Caster Wheel	955	\$3.95	\$3.95
2	2	Pololu	Motor Bracket	2676	\$7.95	\$7.95
6	6	UCI	3D Printing		\$20.00	\$20.00
0.5	0.5	UCI	Laser Cutting		\$16.00	\$16.00
1	1	UCI	Italian Poplar 16"x10" (6mm thick)		\$2.78	\$2.78
				Subtotal		\$281.79
				*Tax rate:	7.75%	\$21.84
				TOTAL ORDER PRICE		\$303.63

Appendix C: Arduino Code

```
//Team Name: CrustyCrab
//Team Members: Ashley Roselynn Vincent, Richard Quoc Ngyuen, Jennifer
Martinez, Sheily Serrano,
//Danny Bao, Joshua J. Lam, Jordan Rui Chan
//Date of Last Edit: 3/14/25
//Summary: This code is written to make our rover, the CrustyCab, follow
the black line path on the track,
// then color track the blue object, move to the object and grab and lift
the object when it reaches the target distance
#include "CytronMotorDriver.h"
CytronMD motorLeft(PWM_DIR,3,4); //52
CytronMD motorRight(PWM_DIR,6,7); //67

#include <Pixy2.h> // Allows communication with the Pixy2 camera
#include <Servo.h>

//do not touch pin #'s and motor #'s 125
const int SensorMiddle = 8;
const int SensorLeft = 4;
const int SensorRight = 9;

int var = 1; //Initialize the program in case 1

// Create objects for Pixy2 camera, lift, and claw servo
Pixy2 pixy;
Servo claw;
Servo lift;

const int CENTER_X = 157.5; // The x-position that represents the
center of the camera's view
const int TARGET_WIDTH = 100; // The object's width when it is close enough
to grab
const float KP = 1; // Proportional gain for turning control (used
for feedback)
const int MAX_SPEED = 255; // Maximum motor speed
const int MIN_SPEED = 50; // Minimum motor speed (ensures the robot
moves instead of stalling)
const int TURN_THRESHOLD = 10; // How close the object must be to the
center before moving forward
```

```

const int TURN_SPEED = 255; // Speed for turning

//PID Constants & Tuning
/*
Note to self:
Increasing Kp = more correction, may wobble/oscillate if too high
Increasing Ki = long term correction, may wobble if too high
Increasing Kd = smoother turns, may not be receptive if too high
LEAVE LARGE DIFF BETWEEN Kp and Kd during testing
Current best settings:
{Kp = 160, Ki = 0.15, Kd = 55, baseSpeed = 150} - completes preliminary
track
Attempt for higher cases of base speed, and increment Kp by 10-20, Kd by
5-10 and Ki by 0.05 (rarely)
ADD SPIN CASE TO ACCOUNT FOR BACKWARDS MOVEMENT - ADD AFTER CODE MESH OR
BETWEEN SWITCH CASE?

*/
float Kp = 160; //Proportional gain
float Ki = 0.15; //Integral gain
float Kd = 55; //Derivative gain

int recentError = 0;
float integral = 0;

const int baseSpeed = 150;

void setup() {
  // put your setup code here, to run once:
  pinMode(SensorMiddle, INPUT);
  pinMode(SensorLeft, INPUT);
  pinMode(SensorRight, INPUT);

  Serial.begin(115200);
  claw.attach(9); // Attach the servo motor for the claw to digital pin 9
  claw.write(90); // Start with the claw open
  lift.attach(10); // Attach the servo motor for the lift to digital pin 10
  lift.write(90); // Start with the Lift level
  pixy.init(); // Initialize Pixy2 camera

```

```

}

void loop() {
    // put your main code here, to run repeatedly:
    // !digitalRead indicates when it sees white, otherwise black
    // must see white to keep traveling linearly without stops, cannot
change 'not' statements
    // havent figured out proper startup without manual switch

    bool detectLine = !digitalRead(SensorMiddle) ||
!digitalRead(SensorLeft) || !digitalRead(SensorRight);

    // detection of black lines = stops
    if (!detectLine) {
        motorLeft.setSpeed(0);
        motorRight.setSpeed(0);
        pixyTracking();
        return; // truth statement awaiting final line detection
    }

    int ERROR = 0;
    //polarities of motor were switched, Left <=> Right became swapped in
1, -1 respectively
    if (digitalRead(SensorMiddle)) ERROR = 0; //straight
    if (digitalRead(SensorLeft)) ERROR = 1; //left
    if (digitalRead(SensorRight)) ERROR = -1; //right
    if (digitalRead(SensorLeft) && digitalRead(SensorRight)) ERROR = 0;

    //calculations based on PID theory, refer to PID formula and code
integration
    // integral needs to add up error over time, whilst derivative
calculates the existing error and changes speed as necessary
    integral += ERROR;
    int derivative = ERROR - recentError;
    float correction = (Kp * ERROR) + (Ki * integral) + (Kd * derivative);
    recentError = ERROR;

    int leftSpeed = baseSpeed - correction; //following same logic as above
    int rightSpeed = baseSpeed + correction;
    leftSpeed = constrain(leftSpeed, -255, 255); //max constraints for
motor

```

```

    rightSpeed = constrain(rightSpeed, -255, 255);

    motorLeft.setSpeed(leftSpeed);
    motorRight.setSpeed(rightSpeed);

    delay(10); //good delay amt, do not change
}

void pixyTracking() {
    if (error > TURN_THRESHOLD){
        pivotLeft (speed1);
    }
    else if (error < -(TURN_THRESHOLD)){
        pivotRight(speed1);
    }
    else {
        if(object_width >= TARGET_WIDTH){
            grabObject();
        }
        else{
            moveForward1();
        }
    }
}

else {
    pivotRight(75);
}

// Looping code
void loop()
{
    trackObject();
}

// Function to stop motors
void stopMotors() {
    // TODO: Set both motor speeds to zero

```

```

    motorRight.setSpeed(0);
    motorLeft.setSpeed(0);
}

// Function to stop the robot and close the claw
void grabObject() {
    stopMotors();
    delay(1000);
    claw.write(150);
    delay(1000);
    lift.write(135);
}

// Function to move forward
void moveForward() {
    motorRight.setSpeed(150);
    motorLeft.setSpeed(150);
}

// Function to move forward during color tracking
void moveForward1() {
    motorRight.setSpeed(200);
    motorLeft.setSpeed(200);
}

// Function to turn left
void pivotRight(int speed) {
    motorRight.setSpeed(speed);
    motorLeft.setSpeed(-speed);
}

// Function to turn right
void pivotLeft(int speed) {
    motorRight.setSpeed(-speed);
    motorLeft.setSpeed(speed);
}

// Function to track and approach an object using Pixy2
void trackObject() {
    pixy.ccc.getBlocks();
    if (pixy.ccc.numBlocks >= 1){
        int object_x = pixy.ccc.blocks[0].m_x;

```

```
int object_width = pixy.ccc.blocks[0].m_width;  
int error = object_x - CENTER_X;  
int output = KP * abs(error);  
int speed1 = constrain(output, MIN_SPEED, MAX_SPEED);
```

Appendix D: References

Agrobot. (2023). *AI-Powered Agricultural Rovers for Food Production*. Retrieved from <https://www.agrobot.com>

Nuro. (2022). *Self-Driving Vehicles for Food & Grocery Delivery*. Retrieved from <https://www.nuro.ai>

Smith, J. (2021). *The Role of Robotics in Food Industry Automation*. Journal of Robotics and AI, 12(3), 45-60.

Starship Technologies. (2023). *Autonomous Delivery Robots in Urban Areas*. Retrieved from <https://www.starship.xyz>